



Маркетплейс

Полная техническая и функциональная документация

Микросервисная платформа электронной коммерции

handmade-товары · цифровые товары · онлайн-курсы (LMS)

FastAPI · React · PostgreSQL · Yandex Cloud

Версия документа 1.0 · Июль 2026

Оглавление

1. Обзор проекта
2. Технологический стек
3. Архитектура
4. Микросервисы и их границы
5. Модель данных
6. Слой бизнес-логики
7. API
8. Фоновые задачи (Celery)
9. Внешние интеграции
10. Фронтенд
11. Кабинет продавца (поддомен)
12. Инфраструктура и деплой
13. Безопасность
14. Функциональные возможности
15. Рекомендации: что есть и что улучшить

1. Обзор проекта

Маркетплейс — это полнофункциональная многосторонняя платформа электронной коммерции (multi-vendor marketplace) с уклоном в **handmade / крафтовые товары**. Помимо классической продажи физических товаров платформа поддерживает два дополнительных бизнес-направления: **цифровые товары с мгновенной доставкой** (файлы, лицензии) и **встроенную LMS** — продажу и прохождение онлайн-курсов с видео, уроками, тестами и сертификатами.

Проект реализует полный жизненный цикл торговли: витрину и каталог, корзину и оформление заказа, онлайн-оплату с фискализацией (54-ФЗ), доставку через российских логистических операторов, разделение платежей между продавцами, вывод средств, возвраты и споры, поддержку, маркетинговые инструменты (промокоды, акции, продвижение, программы лояльности и рефералов), а также богатую админ-панель и отдельные кабинеты для продавцов.



Целевая аудитория и роли. В системе пять ролей пользователей: **buyer** — покупатель, **seller** — продавец, **support** — поддержка, **moderator** — модератор, **superadmin** — суперадмин. Права внутри админки дополнительно гранулируются через RBAC (набор разрешений на пункты меню).

2. Технологический стек

Слой	Технологии
Backend	Python 3.12, FastAPI , SQLAlchemy 2 (async), Pydantic v2 / pydantic-settings, Alembic (миграции), Uvicorn
База данных	PostgreSQL 17 (Yandex Managed, пулер :6432), драйвер asyncpg
Кэш / очереди	Redis / Valkey 8, Celery (worker + beat) для фоновых задач
Поиск	Meilisearch (опционально; fallback на ILIKE по БД)

Frontend	React 18 + TypeScript, Vite, Ant Design 5 , Zustand (состояние), React Router 6, i18next (RU/EN), Recharts, HLS.js
API-шлюз	Kong 3.6 (Postgres-backed) — единый edge, маршрутизация по префиксам пути
Инфраструктура	Yandex Cloud : Network Load Balancer, Compute Instance Groups, Managed PostgreSQL/Redis, Object Storage (S3), Cloud DNS, Postbox (SMTP), Container Registry
IaC / деплой	Terraform (вся инфраструктура) + Ansible (выкатка контейнеров, настройка Kong), Docker
Наблюдаемость	Prometheus + Grafana, Sentry (опционально), Prometheus-метрики на каждом сервисе
Платежи / фискализация	ЮKassa (YooKassa) + встроенная фискализация чеков 54-ФЗ
Доставка	СДЭК, Ozon, Yandex Delivery, Почта России

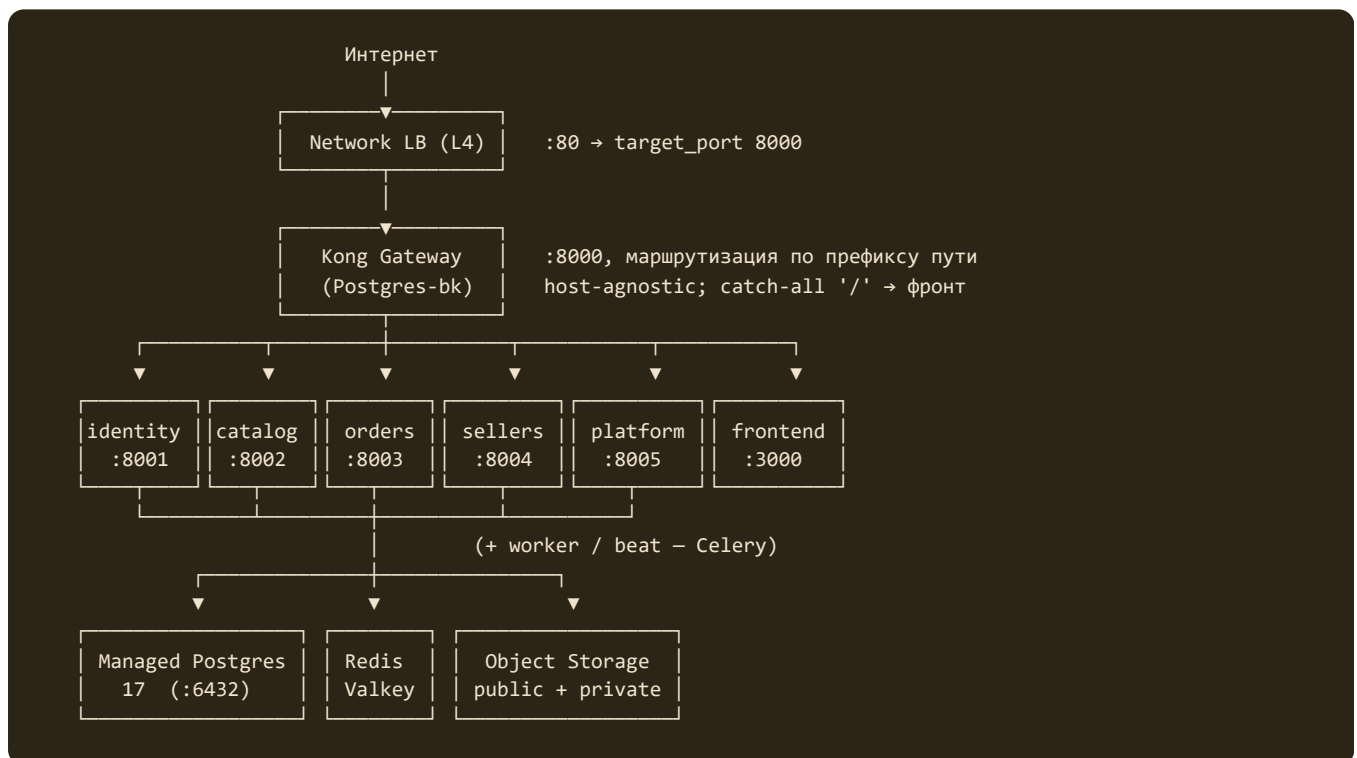
3. Архитектура

Изначально проект был **монолитом на FastAPI** (~16К строк). Была проведена поэтапная (staged) миграция:

- **Этап 1:** перевод с MySQL на PostgreSQL, адаптация под инфраструктуру Yandex Cloud.
- **Этап 2:** распил на **микросервисы поверх общей БД** (shared-database microservices) — 5 независимо разворачиваемых FastAPI-сервисов, разделённых по доменам, плюс Celery worker/beat.

Ключевое архитектурное решение. Выбрана модель «сервисы на общей Managed PostgreSQL», а НЕ классические микросервисы «БД на сервис». Общий код вынесен в устанавливаемую библиотеку `shared/app` (пакет `app`) — модели, схемы, сервисы, конфигурация переиспользуются всеми сервисами без дублирования. Каждый сервис — свой Docker-образ и своя запись маршрута в Kong.

Топология production



Фронтенд — отдельный статический контейнер (nginx), обращается к API по **относительному** `/api/v1` (тот же origin через Kong), поэтому CORS не требуется. Kong отдаёт catch-all маршрут `/` на фронтенд, а более длинные префиксы (`/api/v1/*`, `/health`, `/uploads`) выигрывают и уходят на backend.

4. Микросервисы и их границы

38 роутеров распределены строго по одному между пятью сервисами; префиксы пути принадлежат ровно одному сервису, что позволяет Kong маршрутизировать без коллизий.

Сервис	Порт	Домен / префиксы Kong
identity	8001	Аутентификация, пользователи, 2FA, безопасность: <code>/auth</code> , <code>/users</code> , <code>/2fa</code> , <code>/security</code>
catalog	8002	Товары, магазины, категории, отзывы, курсы (продажа), избранное, рекомендации, витрина: <code>/products</code> , <code>/shops</code> , <code>/reviews</code> , <code>/categories</code> , <code>/courses</code> , <code>/learning</code> , <code>/catalog</code> , <code>/home</code> , <code>/uploads</code>
orders	8003	Корзина, заказы, суб-заказы, возвраты, споры (покупатель), подписки на товары, адреса, вишлисты, цифровая библиотека, доставка: <code>/cart</code> , <code>/orders</code> , <code>/returns</code> , <code>/disputes</code> , <code>/sub-orders</code> , <code>/product-subscriptions</code> , <code>/addresses</code> , <code>/library</code>
sellers	8004	Весь кабинет продавца, инструменты, склад, тарифы/подписки, продвижение, споры продавца, академия, индивидуальные заказы: <code>/seller</code> , <code>/subscription</code> , <code>/promotions</code> , <code>/upload</code>
platform	8005	Уведомления, чат, поддержка, подарочные сертификаты, лояльность, валюты, вся админ-панель : <code>/notifications</code> , <code>/chat</code> , <code>/support</code> , <code>/gift-certificates</code> , <code>/loyalty</code> , <code>/currencies</code> , <code>/admin</code>
worker / beat	—	Celery: периодические и отложенные задачи (тот же образ, команды <code>worker</code> / <code>beat</code>)

Локально топология зеркалируется через `gateway/nginx.conf` (аналог маршрутов Kong) и `docker-compose.yml` (gateway + 5 сервисов + worker/beat + migrate + Postgres/Redis + frontend).

5. Модель данных

Единая схема из **85 таблиц** и ~33 перечислений (enum). Enum-типы намеренно объявлены как **VARCHAR + CHECK** (native_enum=False), чтобы избежать проблем PostgreSQL с повторным созданием/изменением типов. Ниже таблицы сгруппированы по доменам.

Пользователи, магазины, доступ

user, shop, shop_member, seller_verification, seller_requisites, seller_plan, seller_subscription, withdrawal_account, verification_code, password_reset_token, audit_log, feature_flag, settings

Каталог и контент

category, product, product_image, product_variant, product_co_purchase, attribute, product_attribute_value, product_question, favorite, product_view, homepage_banner

Цифровые товары и LMS (курсы)

digital_asset, entitlement, course, course_module, course_lesson, lesson_progress, quiz_attempt, certificate, academy_course, academy_lesson, academy_progress

Заказы, платежи, финансы

cart_item, order, order_item, sub_order, payment, fiscal_receipt, transaction, delivery_info, return_request, payout_request, balance_transaction, installment_plan, withdrawal_account

Отзывы, вопросы, споры, поддержка

review, review_reply, review_vote, review_photo, product_question, report, dispute, dispute_message, support_ticket, support_message

Маркетинг и лояльность

coupon, seller_coupon, promotion, ad_wallet_transaction, promo_rule, bundle, bundle_item, flash_sale, campaign, referral, referral_reward, gift_certificate, promo_balance_transaction, loyalty_tier, paid_feature

Коммуникации и прочее

notification, chat_thread, chat_message, chat_template, sms_log, product_subscription, shop_follow, currency_rate, address, wishlist_collection, wishlist_item, stock_movement, custom_request, custom_message

Замечание. Модель насыщена кросс-доменными внешними ключами (~40 связей), поэтому и был выбран вариант общей БД: разрезать её на изолированные базы было бы дорого и рискованно на текущем этапе.

6. Слой бизнес-логики (50 сервисов)

Вся доменная логика вынесена в `shared/app/services` — тонкие роутеры вызывают сервисы. Ключевые из них:

Продажи и деньги

- **payment_service** — оплата через ЮKassa
- **fiscal_service** — чеки 54-ФЗ
- **commission_service** — комиссии платформы
- **suborder_service** — расщепление заказа по продавцам
- **payout_service** — выплаты продавцам
- **order_reversal_service** — отмены/возвраты денег
- **return_service** — возвраты товаров
- **bnpl_service** — рассрочка (BNPL)
- **delivery_service** — логистические операторы
- **stock_service / label_service** — склад, ярлыки

Маркетинг

- **promotion_service** — продвижение
- **promo_rules_service** — акции/наборы
- **campaign_service** — рассылки
- **referral_service** — рефералы
- **gift_service** — подарочные сертификаты
- **loyalty_service / loyalty_tier_service** — лояльность
- **currency_service** — мультивалютность
- **subscription_service** — подписки на товары

Каталог и контент

- **search_service** — Meilisearch / ILIKE
- **recommendation_service** — «с этим покупают»
- **rating_service** — рейтинги
- **slug_service** — ЧПУ-адреса
- **image_service** — обработка изображений
- **course_service** — LMS, HLS-упаковка
- **digital_storage_service** — приватная выдача файлов
- **entitlement_service** — доступы к цифровым товарам
- **certificate_service** — сертификаты курсов

Платформа и доверие

- **rbac_service** — права в админке
- **verification_service / trust_service** — KYC, доверие
- **moderation_service** — модерация
- **dispute_service / support_service** — споры, поддержка
- **notification_service / email_service / sms_service**
- **analytics_service** — аналитика, когорты
- **audit_service** — журнал действий
- **twofa_service** — 2FA
- **cache_service / settings_service**

7. API

REST API на FastAPI: **347 эндпоинтов** в 29 роутер-модулях, единый префикс `/api/v1`.
Автодокументация OpenAPI/Swagger доступна на каждом сервисе.

Группа роутеров	Назначение
auth, twofa, misc(users)	Регистрация, вход, refresh-токены, восстановление пароля, 2FA, профиль
products, catalog_extra, shops, categories, reviews	Каталог, карточки товаров, магазины, отзывы, вопросы, атрибуты, варианты
courses, digital_library, academy	Продажа/прохождение курсов, цифровая библиотека покупателя, академия продавца
cart, orders, returns_orders, subscription	Корзина, оформление, статусы, возвраты, подписки на товары
disputes, support	Споры по заказам (покупатель/продавец/арбитраж), тикеты поддержки
seller_tools, seller_inventory, seller_extra	Кабинет продавца: товары, склад, аналитика, реквизиты, сотрудники, продвижение
promo, promotions, gifts, loyalty	Промокоды, акции/наборы, продвижение, сертификаты, лояльность
custom_orders	Индивидуальные заказы (запрос → расчёт → предоплата)
notifications, misc(chat/delivery/currencies)	Уведомления, чат, доставка, валюты, баннеры
admin	Вся админ-панель: пользователи, магазины, заказы, финансы, модерация, настройки

8. Фоновые задачи (Celery)

Celery worker + beat выполняют периодические и отложенные операции:

Задача	Расписание	Назначение
auto_mark_delivered	каждый час	Автопометка отгруженных заказов доставленными через N дней
auto_complete_orders	каждый час	Автозавершение доставленных заказов (запуск выплат продавцам)
process_subscription_renewals	ежедневно 03:00	Списание/продление подписок продавцов на тарифы
notify_product_subscriptions	каждые 15 мин	Уведомления «снова в наличии» / «цена упала»

remind_abandoned_carts	ежедневно 12:00	Напоминания о брошенных корзинах
rebuild_recommendations	ночью 04:00	Пересборка рекомендаций «покупают вместе»
settle_promotions	периодически	Закрытие/расчёт кампаний продвижения
support_sla_sweep	периодически	Эскалация обращений при нарушении SLA
loyalty_decay	периодически	Сгорание/пересчёт баллов лояльности
send_marketing_campaign	по триггеру	Отправка маркетинговых рассылок
package_lesson_hls	по триггеру	Упаковка загруженного видео урока в HLS для стриминга

9. Внешние интеграции

Интеграция	Роль в системе
ЮKassa (YooKassa)	Приём онлайн-платежей, возвраты; настраивается через <code>YOOKASSA_SHOP_ID/SECRET</code>
Фискализация 54-ФЗ	Встроенная фискализация чеков ЮKassa: коды НДС/СНО, признак предмета и способа расчёта, поддержка агентской схемы маркетплейса
СДЭК, Ozon, Yandex Delivery, Почта России	Расчёт доставки и оформление отправок через российских операторов
SMSC.ru	SMS-уведомления и коды подтверждения (<code>sms_service</code> , лог в <code>sms_log</code>)
SMTP / Yandex Postbox	Транзакционные письма (сброс пароля, уведомления); при пустом хосте письма пишутся в лог
Yandex Object Storage (S3)	Два бакета: публичный (картинки товаров, per-object public-read) и приватный (цифровые товары, HLS, KYC — только presigned-выдача)
Meilisearch	Полнотекстовый поиск (опционально; иначе fallback на ILIKE)
Sentry	Трассировка ошибок (опционально)
Prometheus + Grafana	Метрики каждого сервиса; дашборд встраивается в админку iframe-ом

Все интеграции сделаны **опциональными и деградируемыми**: при отсутствии ключей система переключается на локальные заглушки (файлы на диск, письма в лог, поиск по БД), что упрощает локальную разработку.

10. Фронтенд

SPA на React 18 + TypeScript + Ant Design 5, ~16K строк, около **96 экранов**. Состояние — Zustand (auth, cart, compare, currency). Локализация RU/EN через i18next. Ленивая загрузка (code-splitting) всех страниц.

Дизайн-язык «craft / wood»

Тёплая палитра ремесленного маркетплейса: акцент `#b45309` (терракота/амбра), успех `#4d7c0f` (мох), заголовки `#5b3a1e` (орех), фон-пергамент `#f7f1e8` , скруглённые углы, лёгкая текстура дерева. Логотип  Маркетплейс.

Экраны по ролям

Раздел	Кол-во	Примеры
Витрина (catalog)	6	Главная, каталог, карточка товара, магазин, сравнение, проверка сертификата
Аутентификация	5	Вход, регистрация, сброс/восстановление пароля, подтверждение e-mail
Покупатель (buyer)	22	Корзина, checkout, заказы, возвраты, споры, чат, избранное, коллекции, адреса, лояльность, сертификаты, рефералы, обучение, покупки, подписки, 2FA
Продавец (seller)	23	Дашборд, товары, заказы, склад, импорт CSV, аналитика, отзывы, возвраты, промокоды, акции, продвижение, выплаты, тариф, реквизиты, сотрудники, KYC, академия, споры, курсы, индивидуальные заказы
Админка (admin)	33	Дашборд, пользователи, магазины, верификации, товары, модерация, категории, заказы, выплаты, чеки, купоны, баннеры, рассылки, сертификаты, лояльность, тарифы, платные возможности, академия, валюты, жалобы, feature-flags, SMS, аудит, реконсиляция, когорты, метрики, настройки
Поддержка (support)	2	Стол поддержки, арбитраж споров

Навигация

Админка и кабинет продавца используют группированное **боковое меню (sider)** из сворачиваемых секций; в админке пункты фильтруются по RBAC-правам пользователя. Меню покупателя — сгруппированный выпадающий список у аватара.

11. Кабинет продавца на отдельном поддомене

Кабинет продавца вынесен на **отдельный вход на поддомене** `seller.<домен>` с собственным sider-меню в стиле админки (6 групп: Обзор / Товары / Продажи / Маркетинг / Финансы / Магазин).

Как устроено (Вариант 1 — единый бандл, host-aware)

- Тот же фронтенд-контейнер и бандл обслуживают и витрину, и поддомен. На старте SPA проверяет `window.location.hostname`; если он начинается с `seller.` — рендерится отдельное приложение продавца (свой логин + `SellerLayout`) вместо витрины.
- Внутри кабинета — корневые URL: `seller.<домен>/products`, `/orders` и т.д. Старые ссылки `<домен>/seller/*` редиректят на поддомен.
- Отдельный вход отклоняет не-продавцов; авторизация в localStorage пер-origin, поэтому вход на поддомене независим от витрины.

Инфраструктура — только DNS

Kong catch-all `/` и `/api/v1/*` маршрутизируются по пути (host-agnostic), поэтому тот же контейнер и backend уже обслуживают поддомен. Управление DNS вынесено в Terraform (`infra/terraform/dns.tf`), включается переменной `domain_name`: создаётся зона Cloud DNS + A-записи апекса и `seller.` на один и тот же IP балансировщика. Роль `dns.editor` у CA Terraform.

На будущее (Вариант 2). Код изолирован (`SellerApp` — самостоятельное дерево роутов), поэтому переезд на отдельную сборку/контейнер с собственным маршрутом Kong по хосту будет правкой сборки, а не переписыванием.

12. Инфраструктура и деплой

Terraform

Вся инфраструктура описана в `infra/terraform`: сеть (public/private подсети), Managed PostgreSQL 17 и Redis/Valkey, Compute Instance Groups для Kong и backend, Network Load Balancer, target-группы, Object Storage (2 бакета + статический ключ), Postbox (SMTP), Cloud DNS, а также автогенерируемый Ansible-инвентарь `hosts.ini` со всеми секретами и адресами.

Ansible

Плейбуки в `infra/ansible`: `deploy-all.yml` (CA БД → сервисы+миграции+Kong → фронтенд одной командой), `deploy-services.yml` (5 сервисов + worker/beat + маршруты Kong), `deploy-frontend.yml` (независимый контейнер фронта + catch-all Kong), `kong-setup.yml` (bootstrap Kong на Postgres), `observability.yml` (Prometheus + Grafana на бастине).

CI/CD

GitHub Actions: `ci.yml` (компиляция backend + pytest, сборка фронта) и `deploy.yml` (сборка и пуш 6 backend-образов + фронтенда в Yandex Container Registry, опциональный передеплой Ansible). Скрипт `build-and-push.sh` собирает и пушит 7 образов.

Env-контракт. Ansible инжектит в контейнеры строгий набор переменных: `DATABASE_URL` и `DB_*` (пулер :6432), `REDIS_URL`, `SECRET_KEY` / `REFRESH_SECRET_KEY` / `JWT_ALGORITHM`, `S3_*` и `SMTP_*`, `FRONTEND_URL`. Миграции запускаются одноразовым контейнером (`migrate`), образ backend диспетчеризует команды `migrate | web | worker | beat | seed`.

13. Безопасность

- **JWT-аутентификация** с отдельными секретами для access- и refresh-токенов; авто-обновление токена на 401 через перехватчик axios.
- **RBAC** в админке: набор разрешений определяет доступные пункты меню и эндпоинты (`rbac_service`, эндпоинт «моё меню»).
- **Двухфакторная аутентификация (2FA)** — отдельный сервис и экраны.
- **KYC / верификация продавцов** (`verification_service`, `trust_service`): статусы доверия, VIP, документы хранятся в приватном бакете.
- **Разделение хранилищ**: публичные ассеты и приватные (цифровые товары, HLS, KYC) — разные бакеты; приватные отдаются только по presigned-URL, анонимный доступ = 403.
- **Rate limiting** (`core/ratelimit.py`), хеширование паролей bcrypt.
- **Аудит действий** (`audit_log`) для чувствительных операций админки.
- **TLS БД**: поддержка `DB_SSL` + корневой сертификат Yandex для Managed PostgreSQL.

Открытый пункт. На публичном контуре пока **нет HTTPS** на самом edge: NLB работает на L4 (TCP), TLS предполагается терминировать на Kong отдельным шагом (сертификат wildcard `*.<домен>` или SAN с поддоменом). Grafana развёрнута с анонимным доступом — допустимо для демо, требует закрытия в проде.

14. Функциональные возможности

14.1 Покупатель

- Каталог с фильтрами, атрибутами, вариантами товара, сравнением, вопросами к товару.
- Корзина, оформление заказа, несколько адресов доставки, выбор оператора доставки.
- Онлайн-оплата с чеком 54-ФЗ; **рассрочка (BNPL)**.
- Заказы и суб-заказы (по разным продавцам), отслеживание статусов, возвраты, споры.
- **Цифровые товары**: мгновенная выдача файлов, личная библиотека покупок.
- **Онлайн-курсы**: плеер с HLS-видео, уроки, тесты, прогресс, сертификаты с проверкой по коду.
- Избранное, коллекции (вишлисты), подписка на магазины, подписки на товары («снова в наличии»/«цена упала»).
- Программа **лояльности** (уровни/баллы), **рефералы**, **подарочные сертификаты** и промо-баланс.
- **Индивидуальные заказы**: запрос на изготовление → расчёт продавцом → частичная предоплата.
- Чат с продавцом, уведомления, тикеты поддержки, мультивалютность, RU/EN.

14.2 Продавец

- Дашборд, управление товарами, **импорт из CSV**, склад и движения остатков, варианты/атрибуты.
- Заказы и суб-заказы, возвраты, споры со стороны продавца.
- **Аналитика продаж**, отзывы на товары и ответы на них.
- Маркетинг: собственные промокоды, **флеш-распродажи**, акции и наборы (bundle), **платное продвижение** с рекламным кошельком.
- Финансы: вывод средств, тариф и комиссия (подписки продавца), налоговые реквизиты и режим (СНО).
- **Сотрудники магазина** (роли внутри магазина), **KYC / доверие / VIP**.
- **Академия продавца** (обучающие курсы для продавцов), конструктор курсов, шаблоны чата и часы работы.

14.3 Модератор / Администратор

- Управление пользователями и магазинами (360°-карточки), верификация продавцов.
- Модерация товаров и очередь модерации, категории, отзывы, жалобы.
- Заказы, **выплаты, фискальные чеки, реконсиляция** платежей.
- Маркетинг платформы: купоны, баннеры, рассылки, сертификаты, уровни лояльности, рефералы.
- Настройки платформы, тарифы, платные возможности, валюты, **feature-flags**, SMS.
- Аналитика платформы, **когорты и LTV**, метрики Grafana, журнал аудита.

14.4 Поддержка

- Стол поддержки (тикеты с SLA и авто-назначением), **арбитраж споров** между покупателем и продавцом.

15. Рекомендации: что есть и что можно улучшить

Что реализовано сильно

- **Широта функционала:** физические + цифровые товары + LMS, полный цикл денег (оплата → фискализация → расщепление → выплаты → возвраты), богатый маркетинг.
- **Чистая поэтапная миграция** монолита в сервисы на общей БД без переписывания импортов (общая библиотека `app`).
- **Инфраструктура как код** целиком: Terraform + Ansible + CI/CD, воспроизводимый деплой в Yandex Cloud.
- **Продуманная деградация** интеграций (S3/SMTP/поиск/SMS) — удобно для разработки.
- **Разделение публичных/приватных хранилищ** и presigned-выдача цифровых товаров.

Приоритетные улучшения

Инфраструктура и эксплуатация

- **HTTPS/TLS на edge** — сейчас публичный контур по HTTP. Терминировать TLS на Kong (wildcard-сертификат, включая поддомен продавца), включить HSTS.
- **Закрыть Grafana** (сейчас анонимный доступ) и вынести мониторинг за аутентификацию.
- Строгая сборка фронтенда: сейчас прод собирается через `vite build` в обход `tsc` из-за оставшихся ошибок типов — починить типы и вернуть `tsc && vite build`.
- Зафиксировать зависимости фронтенда (нет `package-lock.json` → CI использует `npm install`); перейти на `npm ci` для воспроизводимости.
- Секреты — вынести в управляемое хранилище (Yandex Lockbox / Vault) вместо инъекции через Ansible-inventory.

Архитектура и данные

- Общая БД — узкое место при росте: ввести **схемы-на-сервис** и явные контракты, чтобы позже безболезненно разнести данные (движение к Варианту 2 и для фронта продавца).
- Единая монорепа-библиотека `app` связывает сервисы на уровне кода — по мере роста выделять доменные пакеты с версионированием.
- Добавить нагрузочное тестирование и явные индексы под тяжёлые аналитические запросы (когорты, реконсиляция).

Качество и надёжность

- Расширить автотесты (сейчас минимальный pytest в CI) — контрактные тесты API и e2e ключевых сценариев (оплата, возврат, выдача цифрового товара).
- Идемпотентность платёжных вебхуков и повторных попыток выплат; сверка с ОФД/эквайрингом.
- Централизованное структурированное логирование + трассировка запросов между сервисами (trace-id через Kong).

Продукт и рост (отложенный roadmap)

Уже согласованы к постройке ближайшие пункты: рассрочка (BNPL), индивидуальные заказы, подарочная упаковка/сертификаты, Академия продавца, расширенные отзывы (видео, бейдж проверенной покупки, галерея). **Отложено** (рекомендуется вернуться после):

- **AI-слой**: чат-бот поддержки, авто-генерация описаний/тегов, семантический/визуальный поиск, персональная лента.
- **Рекламный кабинет продавца** с аналитикой ставок/охвата/ROI, A/B-тесты карточек, прогноз спроса.
- **Live-шопинг** и короткие видео (reels).
- **Платная подписка** покупателя (бесплатная доставка + привилегии) на базе лояльности.
- **Escrow-бейдж** (выплата при завершении заказа $\approx$ escrow — сделать это явным преимуществом).
- **Fulfilment-light** (склад маркетплейса, консолидированная доставка).
- **Мобильные приложения / PWA**.
- **B2B-режим** (прайс-листы, счета, опт) и **трансграничная торговля** (i18n-контент, таможня, мультивалютные расчёты).

UX и мелочи

- Единый onboarding продавца (KYC → реквизиты → тариф) как пошаговый мастер.
- Сквозной поиск и подсказки на витрине через Meilisearch (включить в проде).
- Доступность (a11y) и мобильная адаптация ключевых экранов чекаута и кабинета продавца.